

PROJECT DOCUMENTATION

ERP Connector

Connector Scaling: Planning, Architecture & Roadmap

TABLE OF CONTENTS

1. Purpose of This Document
2. Current System -What Already Exists
3. Problem Statement -Connector Scaling
4. How Connector Scaling Works Today
5. Evidence From Testing -What Works and What Doesn't
6. The Next Phase -Mapping UI
7. UI/UX Design
8. Backend Design
9. Frontend Design
10. Technology Stack
11. Component Summary -Files, New and Existing
12. Open Design Questions
13. Roadmap
14. Conclusion

1. Purpose of This Document

This document records the full planning and scaffolding of the ERP Connector's Connector Scaling initiative: what already exists in the current system, why the scaling problem matters, what has been tested and learned so far, and the design of the next phase -a Mapping UI that allows a non-technical user to onboard a new ERP without ever seeing raw data or code.

This is a planning document, not a finished specification. Sections 7 through 12 record design decisions made so far and questions still open, so that development can begin from a shared, written understanding rather than informal discussion.

2. Current System -What Already Exists

The ERP Connector is a working microservice that provides one consistent API for retrieving and updating data across Xero and QuickBooks Online. Before describing the scaling initiative, it is important to record what is already built and proven, since Connector Scaling is designed to extend this system, not replace it.

2.1 Core Capabilities Already Working

- Unified REST endpoints for invoices, bills, contacts, accounts, and payments, functioning identically against both Xero and QuickBooks Online
- Fully automated OAuth login and background token renewal for both platforms -no manual token handling required after initial one-time setup
- A configuration-driven field mapping engine (`utils/field_mapper.py`) that converts each platform's raw response into one standard schema
- Automatic retry handling for transient network failures, and per-tenant rate limiting matched to each platform's published limits
- An automated test suite of 119 tests, covering every operation against both platforms end to end

2.2 The Manual Bottleneck This Initiative Addresses

Adding support for a new accounting platform today requires a developer to manually author a YAML mapping file, translating every field in that platform's raw API response into the connector's standard schema. This is not the largest part of onboarding a new platform -building the actual API connection remains the larger share of the work -but it is a distinct, repeatable task well suited to acceleration.

3. Problem Statement -Connector Scaling

“Leverage LLM-based intelligent schema mapping to accelerate onboarding of new connectors, minimizing manual mapping effort and enabling faster integration.”

This statement was evaluated directly against a real, working implementation rather than assumed to be true. The findings from that evaluation shape the design decisions recorded in this document.

4. How Connector Scaling Works Today

A command-line tool (tools/generate_mapping.py) already exists and is functioning. Given a raw sample response from a new ERP, it produces a draft YAML mapping file by prompting an LLM (Google Gemini) to match each raw field against the connector's standard schema.

- The standard schema fields are read directly from the connector's existing Pydantic models (models/schemas.py), so the tool's target format can never drift out of sync with the actual system
- For any field that cannot be filled by a direct rename, the tool is instructed to mark it NEEDS_REVIEW with a stated reason, rather than guess
- Output is always a draft. Nothing is applied automatically to the running system -a human reviews and completes the file before it is used

5. Evidence From Testing -What Works and What Doesn't

The tool was tested against a sample third-party ERP format, used strictly as a diagnostic exercise, not as a real integration. This testing produced concrete, specific findings rather than general impressions.

5.1 What the Tool Handled Correctly

- Plain field renames (identifiers, dates, currency codes) were mapped correctly and consistently
- Fields requiring unit conversion (e.g. currency amounts) were correctly flagged as needing review, rather than guessed

5.2 What the Tool Got Wrong

Three specific classes of error were identified, all sharing the same underlying pattern: the tool treated a field as a safe, direct rename when the field name matched, but the underlying value's format or structure did not.

Field Type	What Went Wrong	Why It Matters
Category/type fields	Raw value used different casing than the standard schema expects	Would silently produce incorrect data with no error raised
Status/state fields	Raw platform-specific values were not translated to the connector's standard status vocabulary	Same silent-failure risk as above
Nested list fields	Raw list items used entirely different field names internally; a rename alone could not work	Most severe case -produces structurally invalid output, not just wrong values

Conclusion: the tool reduces manual effort for the majority of fields, but its judgment about which fields are “safe” cannot yet be fully trusted. This finding directly shapes the design of the Mapping UI described in the following sections -specifically, the decision to show every field's actual sample value during review, not only the fields the tool itself flagged.

6. The Next Phase -Mapping UI

The next phase of Connector Scaling is a user interface that allows a person with no coding or JSON knowledge to review, correct, and confirm a field mapping produced by the LLM tool, resulting in a working connector for a new ERP without that person ever touching a configuration file or writing code.

6.1 Scope Boundary

Building the underlying connection to a new ERP -authentication, API endpoints, and pagination handling -remains a task for a developer and is out of scope for this UI. This interface addresses only the field-mapping stage that follows an already-working connection.

7. UI/UX Design

The interface is designed around one governing rule: the user must never see raw JSON, YAML, or code, at any point in the workflow.

7.1 Screen A -Start

The user selects an already-connected ERP and the type of data being mapped (Invoices, Bills, Customers & Suppliers, or Accounts), described in plain business language. A sample is fetched automatically in the background; the user takes no action to provide one.

7.2 Screen B -Match the Fields

A two-column list pairs each standard field (“Invoice Number,” “Invoice Date,” “Amount”) with the system's best guess at the matching field from the new platform, shown as a plain label. The user confirms each guess or selects a different one from a dropdown of that platform's actual field names.

7.3 Screen C -Teach It What Each Status Means

Triggered automatically whenever a field is detected to contain a fixed set of category-like values (informed directly by the testing finding in Section 5.2). The interface lists the distinct raw values found and asks the user to match each one to the connector's standard equivalent via dropdown -for example, matching a platform-specific status code to “Authorised,” “Draft,” or “Voided.”

7.4 Screen D -Match the Line Item Fields

The same matching pattern as Screen B, applied specifically to the fields nested inside each invoice or bill's line items -directly addressing the nested-list failure identified in testing.

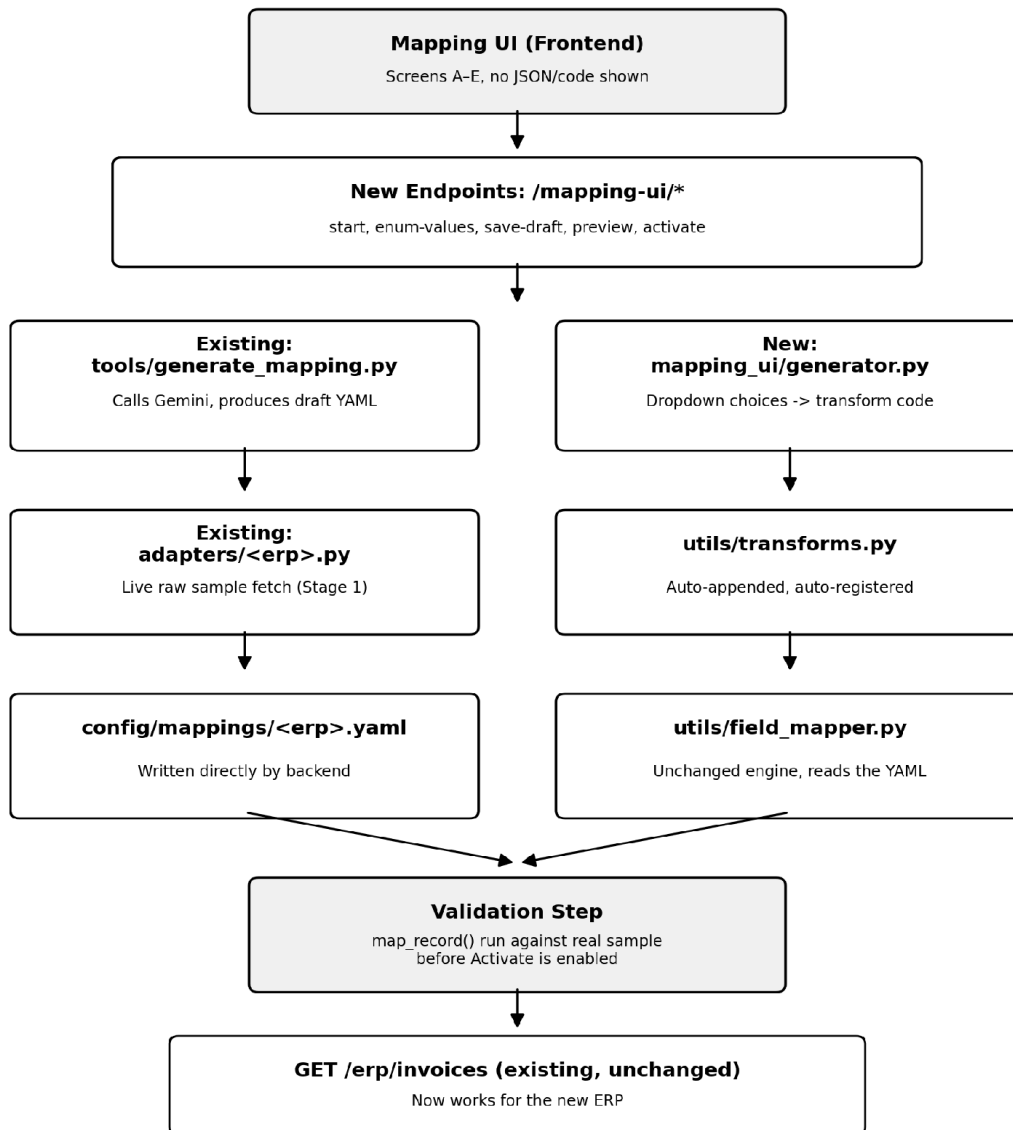
7.5 Screen E -Preview & Activate

A single formatted example record (e.g. “Invoice #1234, Date: June 8, Amount: \$1,623.75, Status: Authorised”) is shown for final confirmation. The Activate action is only enabled once the system has independently verified the mapping produces a complete, valid result.

8. Backend Design

The backend introduces a small set of new endpoints that sit between the UI and the existing, unmodified tooling -the UI never calls the LLM, reads YAML, or writes Python code directly.

Connector Scaling — Backend Architecture



8.1 New Endpoints

POST /mapping-ui/start *[New]* Fetches a live sample from the selected ERP's existing adapter, runs the existing mapping tool, returns a UI-friendly draft

GET /mapping-ui/enum-values *[New]* Scans the fetched sample and returns the distinct values found for a category-like field, powering Screen C

POST /mapping-ui/save-draft *[New]* Writes the real YAML mapping file, and generates and registers any required transform code from the user's Screen C/D choices

POST /mapping-ui/preview *[New]* Runs the real normalization engine against the real sample and returns one formatted record for Screen E

POST /mapping-ui/activate *[New]* Final validation and go-live confirmation for the new mapping

8.2 The Code Generator -the One Substantial New Backend Component

Screen C and D collect the user's choices as simple lookup tables (raw value maps to standard value). A template-based generator converts each completed table directly into a small, predictable transform function, in the same style as the connector's existing transform functions, and registers it automatically. Because this generation is template-based rather than another LLM call, its output is deterministic and testable, not a source of new uncertainty.

8.3 Validation Before Go-Live

Before the Activate action becomes available, the backend independently re-runs the real normalization logic against the real fetched sample and confirms a complete, valid record is produced. This replaces the manual review step a developer would previously have performed by reading code, since no human in this workflow is expected to do so.

9. Frontend Design

The frontend's design responsibility is narrow but important: present every field's real sample value alongside its proposed mapping at all times, not only for fields the system flagged, directly reflecting the finding in Section 5.2 that unflagged fields cannot be assumed safe.

- Plain-language labels throughout -no field ever displayed using its raw technical name alone
- Dropdowns and selection controls only -no free-text code or configuration entry anywhere in the flow
- Clear visual distinction between confirmed mappings and mappings still awaiting user input
- A persistent way to leave and resume a mapping in progress, since reviewing every field for a full resource may not be completed in one sitting

10. Technology Stack

Layer	Choice
Existing backend framework	FastAPI (Python) -new endpoints added to the existing application
LLM provider	Google Gemini (2.5 Flash primary; 2.5 Pro available), already integrated
Mapping storage format	YAML -unchanged, matches the connector's existing format
Frontend framework	To be confirmed -see Section 12
Code generation approach	Deterministic template-based generation, not a further LLM call

11. Component Summary -Files, New and Existing

11.1 Existing, Unchanged

utils/field_mapper.py *[Unchanged]* The generic engine reading whatever mapping file exists for the active ERP

adapters/<erp>.py *[Unchanged]* Still built manually by a developer prior to this UI being used

tools/generate_mapping.py *[Unchanged]* Called internally by the new backend, not modified or duplicated

11.2 New

mapping_ui/routes.py *[New]* The five endpoints described in Section 8.1

mapping_ui/generator.py *[New]* The template-based transform code generator described in Section 8.2

config/mappings/<erp>.yaml *[Generated]* Written automatically by the backend rather than edited by hand

utils/transforms.py *[Appended]* New functions appended and registered automatically, following the existing pattern

12. Open Design Questions

The following were identified during planning as decisions still requiring confirmation before backend and frontend development begins in full.

- Who is the intended user of this interface, and what is their expected level of business/accounting familiarity?
- Is a second approval step required before a mapping goes live, or is a single user trusted end to end?
- How should the system handle a field that genuinely cannot be resolved through simple value-matching -is there a fallback path to a developer, or must the interface handle every case?

- Is this intended as a one-time internal tool or a polished, repeatedly-used product? This materially affects the frontend technology choice and level of investment justified at this stage.

13. Roadmap

Stage	Focus
Stage 1 -Complete	Command-line mapping tool built, tested, and evaluated against real sample data
Stage 2	Close known accuracy gaps in the mapping tool itself (category and nested-list detection) before building UI on top of it
Stage 3	Build backend endpoints (Section 8) against the existing tool, with no frontend yet
Stage 4	Build the frontend screens (Section 7), connected to the Stage 3 backend
Stage 5	Pilot with one real new ERP, onboarded entirely through the UI, validated against a manually-built reference mapping
Stage 6	Resolve open questions from Section 12 based on pilot findings, then decide on broader rollout

14. Conclusion

Connector Scaling is grounded in a working system and a working first tool, evaluated honestly rather than assumed to succeed. That evaluation surfaced specific, addressable weaknesses rather than a general uncertainty, and those findings directly shaped the design of the Mapping UI described in this document -particularly the decision to always show real sample values during review, and to treat category and nested-list fields as inherently requiring user confirmation. The next concrete step is resolving the open questions in Section 12, followed by closing the known accuracy gaps in the underlying mapping tool before backend and frontend development begins in earnest.